EXPERT OPERATIONS REFERENCE

Database Reliability & Platform Engineering

Day-to-Day Operations, Activities & Monitoring

Scope	What a DBRE / Database Platform Engineer actually does each day — across reliability, performance, change, backup/DR, capacity, security and provisioning — and what they monitor.
Part 1	The DBRE / Platform Engineer role — operating cadence, activity domains, and the monitoring stack.
Part 2	Bringing AI into database & platform operations — assistive, analytical and agentic tiers, with guardrails.
Part 3	Operating PostgreSQL / MySQL at scale, as code, with AI-assisted ops — the full GitOps loop and fleet monitoring.

The discipline in one sentence: **treat the data tier like a product** — define its reliability targets, run it through software and automation rather than manual toil, and let AI accelerate the judgment-heavy work while humans keep control of consequential actions.

Practitioner reference • Tooling examples are illustrative; adapt to your environment and versions.

Part 1 — The DBRE / Database Platform Engineer

A Database Reliability Engineer applies **SRE discipline to stateful data systems**. You own the availability, performance, security, cost and full lifecycle of the data tier, and you deliver that through code and automation rather than hands-on toil. You sit as a peer to SRE and platform teams and act as an *enabler* for application teams — giving them a safe, self-service "paved road" to databases rather than being a manual gatekeeper.

> **The five operating principles**
> 1) **Everything as code** — infrastructure, configuration and schema live in Git. 2) **Eliminate toil** — anything done manually twice becomes automation. 3) **SLO-driven** — reliability is a measured target with an error budget, not a vibe. 4) **Blameless operations** — incidents produce learning and systemic fixes, not blame. 5) **Capacity & cost are first-class** — you engineer for growth and spend, not just uptime.

1.1 The daily operating cadence

A DBRE day is a blend of **reactive operations** (keeping the fleet healthy) and **proactive engineering** (building the automation that removes tomorrow's toil). A representative rhythm:

Start of day — the health sweep (~30–45 min)

- Triage overnight alerts and the on-call handover; close noise, action anything real.
- Check SLO dashboards and **error-budget burn** — is any service trending toward breach?
- Verify **replication health and lag** across the fleet, and that standbys are streaming.
- Confirm **backups completed** and that the latest restore test passed (a backup you haven't restored is a hope, not a backup).
- Scan capacity & cost anomalies — disk growth, connection saturation, runaway spend.
- Review the change calendar — what migrations / upgrades land today.

Core hours — engineering & collaboration

- Build and improve automation: IaC modules, operators, migration tooling, self-service.
- Review **schema-change pull requests** from app teams; gate risky migrations.
- Performance investigations: slow queries, plan regressions, lock contention.
- Capacity planning and right-sizing; cost optimisation work.
- Project work: HA improvements, DR automation, observability, platform features.

Recurring & on-call

- **Weekly**: reliability review (SLOs, incidents, error budget), postmortem follow-ups, patch/upgrade planning.
- **Periodic**: DR game-days and failover drills, access reviews, capacity forecasts, version-EOL planning.
- **On-call**: own the data-tier incident lifecycle — detect, mitigate, communicate, learn.

1.2 The activity domains (what the work actually is)

Availability & HA	Design and run replication topologies; automate failover (Patroni for Postgres, Orchestrator for MySQL); monitor standby lag and quorum; run failover game-days so the automation is proven before a real outage, not during one.
Performance engineering	Analyse slow queries and plan regressions; design indexing strategy; tune autovacuum (PG) and InnoDB (MySQL); manage connection pooling; partner with app teams on data-access patterns before they become incidents.
Change & schema management	Treat schema changes as versioned, reviewed code; enforce online/low-lock changes (CONCURRENTLY, gh-ost, pt-osc); gate destructive operations; deliver zero-downtime migrations through CI/CD.
Backup & disaster recovery	Orchestrate backups (pgBackRest, Percona XtraBackup); enforce RPO/RTO targets; run automated restore tests and PITR drills; the metric that matters is tested restore time, not backup success.
Capacity & cost	Forecast growth (storage, IOPS, connections, memory); right-size instances; attribute and control cloud spend; set autoscaling and retention policies.
Security & compliance	Enforce least-privilege access and access reviews; manage secrets and rotation; encryption at rest and in transit; audit logging, data masking; produce compliance evidence (SOC 2, GDPR, etc.).
Provisioning & platform	Build the self-service "paved road": golden templates, Kubernetes operators, guardrails so app teams get a database in minutes within policy — without a ticket to you.
Incident response	Triage and mitigate data-tier incidents; coordinate comms; lead blameless postmortems and drive the systemic fixes that stop repeats.

1.3 Monitoring — philosophy, signals and the stack

Good DBRE monitoring follows three rules: **measure what maps to user pain**, **alert on symptoms not causes**, and **page only on actionable, user-impacting conditions** — everything else is a dashboard or a ticket. This keeps the pager quiet enough that an alert always means something.

The signals to track (the database "golden signals")

Signal	What it tells you	Representative metrics
Availability	Can clients connect and is the node serving its role	up, role (primary/standby), accepting connections
Latency	User-visible slowness	query p50/p95/p99, commit latency
Throughput	Load level / demand	QPS / TPS, transactions committed
Errors	Things actively failing	failed connections, deadlocks, replication errors
Saturation	How close to a limit	CPU, memory, IOPS, connections, lock waits, replication lag, disk %

SLIs, SLOs and error budgets

Turn signals into **SLIs** (the measured indicator), wrap them in an **SLO** (the target), and the gap to 100% is your **error budget** — the amount of unreliability you can spend before you must stop shipping risky changes and stabilise.

> **Example SLO**
> "99.9% of read queries on the orders cluster complete in under 50 ms, measured over a rolling 28 days." A 99.9% target leaves ~43 minutes of monthly budget. You then alert on the **burn rate** of that budget (fast burn pages immediately; slow burn opens a ticket) rather than on raw thresholds.

The monitoring stack

- **Metrics**: Prometheus scraping `node_exporter`, `postgres_exporter`, `mysqld_exporter`; Grafana for dashboards; Alertmanager for routing.
- **Query analytics**: `pg_stat_statements` (PG), the performance/sys schema (MySQL), or Percona PMM for both.
- **Logs**: a pipeline (Loki or ELK) for slow-query logs, errors and audit trails.
- **Backups/DR**: monitor backup age, size, duration and last successful restore.
- **Tracing**: where apps support it, tie slow spans back to specific queries.

Engine-specific metrics that earn their place

PostgreSQL

Watch	Why it matters
Connections vs max_connections	Saturation here causes hard outages; pool with PgBouncer
Transaction ID age (datfrozenxid)	Wraparound risk — must vacuum before the horizon; can force shutdown
Replication lag (bytes & seconds)	Standby usefulness for failover and read scaling
Cache hit ratio	Low ratio signals memory pressure / bad plans
Deadlocks & long-running txns	Block vacuum, hold locks, bloat tables
Autovacuum activity & bloat	Unchecked bloat degrades performance and disk
WAL generation rate & checkpoints	Drives archive volume, recovery time and IO spikes

MySQL / MariaDB

Watch	Why it matters
Threads_connected / Threads_running	Connection & concurrency saturation; front with ProxySQL
Replication lag (Seconds_Behind_Source)	Failover safety and read consistency
InnoDB buffer pool hit ratio	Working set vs RAM; misses drive disk IO
Row lock waits & deadlocks	Contention hot-spots in the schema / access pattern
History list length	Long purge backlog from old open transactions
Slow query rate & aborted connects	Direct user-pain and app-side connection issues
Redo log / checkpoint age	Write pressure and crash-recovery time

Example alerting rules (PromQL)

Symptom-based alerts: page on user-impacting / data-loss-risk conditions, ticket on early warnings.

```
# Postgres replica lag over 30s for 5 minutes -> page
- alert: PostgresReplicaLagHigh
  expr: pg_replication_lag_seconds > 30
  for: 5m
  labels: { severity: page }
  annotations: { summary: "Replica {{ $labels.instance }} lagging > 30s" }

# Connection pool nearing the ceiling (>85%) -> ticket
- alert: PostgresConnectionsHigh
  expr: sum(pg_stat_activity_count) by (instance)
        / max(pg_settings_max_connections) by (instance) > 0.85
  for: 10m
  labels: { severity: ticket }

# Backup older than 26h -> page (daily backup SLA breached)
- alert: BackupStale
  expr: time() - pgbackrest_last_backup_timestamp > 26*3600
  labels: { severity: page }
```

Part 2 — Bringing AI into database & platform operations

The governing idea is **augmentation, not autopilot**. AI is extremely good at the judgment-adjacent work that fills a DBRE's day — reading logs, correlating signals, recalling the right runbook, drafting the postmortem — and weak at being trusted with irreversible production actions unsupervised. So you let it accelerate *analysis and drafting*, and you keep humans on the trigger for anything consequential.

> **Non-negotiable guardrails**
> **Read-only by default.** AI tools query telemetry and propose; they do not mutate production without an explicit human approval gate. **Sandboxed writes** run against staging or dry-run first. **Least-privilege** service accounts. **Full audit** of every AI action and suggestion. **Evaluate before trust** — measure suggestion quality before widening autonomy. **Kill switch** always available.

2.1 Three maturity tiers

Adopt AI in order. Each tier earns the trust needed for the next.

Tier 1 — Assistive (copilots that draft and explain)

- **Natural-language diagnostics**: "why is the orders replica lagging?" → the assistant pulls metrics and explains, citing the data.
- **Query-plan explainer**: paste an EXPLAIN plan, get a plain-language read plus index suggestions to evaluate.
- **Runbook copilot (RAG)**: grounded on *your* runbooks and past incidents, it surfaces the right procedure during on-call.
- **Incident & postmortem drafting**: timeline and summary generated from alert + chat + commit history for you to verify.
- **Migration PR assistance**: summarises what a schema change does and flags risky patterns for the reviewer.

Tier 2 — Analytical (AIOps on your telemetry)

- **Anomaly detection** on metrics — learns normal seasonality and flags deviations static thresholds miss.
- **Forecasting** — "this cluster reaches 90% disk in ~19 days"; feeds capacity work before it pages.
- **Alert correlation & noise reduction** — groups a storm of alerts into one probable root cause.
- **Log clustering & summarisation** — collapses millions of lines into the few novel patterns that matter.
- **Query-plan regression detection** — catches a plan that flipped after a stats change or deploy.

Tier 3 — Agentic (guarded automation)

- **Triage agents** that, on an alert, gather context (metrics, logs, recent changes, similar past incidents) and present a diagnosis + proposed action.
- **Auto-remediation** for well-understood, reversible issues — kill a runaway query, recycle a stuck connection pool — behind an approval gate.
- **Self-service provisioning agents** that turn a request into a policy-checked IaC pull request, not direct production changes.
- **ChatOps**: the agent operates inside Slack/Teams where actions are visible, logged and approvable by the team.

2.2 The building blocks

A practical AI-ops stack wires an LLM to your operational data through retrieval and safe tools:

Component	Role in the system
LLM (e.g. Claude API)	Reasoning, explanation, drafting, tool orchestration
RAG + vector store	Ground answers in your runbooks, schema, past incidents — pgvector keeps it in Postgres you already run
Tool / function calling	Lets the model read Prometheus, query stats, fetch logs, run safe read-only SQL
MCP servers	Expose internal tools (metrics, backups, ticketing) to the model over a standard protocol
Policy engine (e.g. OPA)	Decides what an agent may do autonomously vs. what needs approval
Eval harness	Scores suggestion quality and action outcomes over time
Audit + observability	Logs every prompt, suggestion and action; monitors the AI layer itself

Illustrative: a read-only diagnostic tool exposed to the model

The agent observes and proposes; mutating actions (e.g. pg_terminate_backend) sit behind a human gate.

```
# A guarded, read-only tool the agent may call. Note: SELECT-only,
# parameterised, timeout-bounded, and audit-logged.
def get_active_long_queries(min_seconds: int = 60):
    sql = """
        SELECT pid, now() - query_start AS duration, state, query
        FROM pg_stat_activity
        WHERE state = 'active' AND now() - query_start > %s
        ORDER BY duration DESC LIMIT 20;
    """
    with readonly_pool.connection(timeout=5) as conn: # least-privilege role
        rows = conn.execute(sql, (f"{min_seconds} seconds",)).fetchall()
    audit.log("tool:get_active_long_queries", who="agent", rows=len(rows))
    return rows # the model explains; a human approves any kill action
```

2.3 How the day changes, and monitoring the AI itself

With the layer in place, the morning sweep arrives as an AI-written digest ("overnight: 1 replica lag spike, auto-recovered; backups green; cluster X forecast to hit 90% disk in 19 days"). Performance investigations start from a ranked hypothesis instead of a blank dashboard. On-call gets the relevant runbook and similar past incidents surfaced automatically. None of this removes your judgment — it removes the *search cost* that surrounds your judgment.

> **Monitor the AI layer like any other dependency**
> Track **suggestion acceptance rate** (are humans taking the advice?), **anomaly-detector precision/recall** (is it crying wolf?), **action success rate** for any automated remediation, and **latency & cost** of the model calls. Falling acceptance or rising false positives is your signal to retrain, re-ground or roll back autonomy.

Part 3 — Postgres / MySQL at scale, as code, with AI-assisted ops

This is the synthesis: run the relational fleet through **declarative code and GitOps**, operate it with **scale-grade patterns**, and lay the **AI layer from Part 2** over every stage. "As code" means there is no console click that isn't reproducible from Git.

3.1 The layers of "as code"

Layer	Tooling	What lives here
Infrastructure	Terraform / OpenTofu	Instances, clusters, networking, parameter groups, storage
Configuration	Ansible / operator specs	Engine config, users baseline, extensions
Schema & data	Flyway / Liquibase / Atlas / Bytebase	Versioned, reviewed migrations
Lifecycle on K8s	CloudNativePG, Percona Operator, Vitess	Declarative provisioning, failover, backups
Delivery	Argo CD / Flux (GitOps)	Continuous reconciliation of desired state
Policy	OPA / Conftest	Guardrails enforced in CI and at admission

The change flow a migration travels

PR → **CI** (lint · `migrate diff` · AI risk review · plan) → **staging apply** → **canary** (small slice, watch SLOs) → **prod** via GitOps with automated verification and one-command rollback. AI plugs in at the CI review and the canary-watch stages.

Schema as versioned code; CONCURRENTLY avoids a table-blocking lock on a hot table.

```
-- migrations/0042_add_orders_status_idx.sql
-- Reviewed in PR; CI flags non-concurrent index on a large table.
-- DBRE rule: large-table index builds MUST be online.
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_orders_status
    ON orders (status)
    WHERE status <> 'archived'; -- partial index keeps it small
```

3.2 PostgreSQL at scale — the patterns

- **HA & failover**: Patroni + etcd/Consul (or CloudNativePG on Kubernetes) for leader election and automated, tested failover.
- **Connections**: PgBouncer in transaction mode — Postgres handles far more clients than backends, so pooling is mandatory at scale.
- **Read scaling**: streaming replicas with application/proxy read routing.
- **Big tables**: declarative partitioning; Citus for true sharding when a single node is outgrown.
- **Maintenance**: tuned autovacuum, wraparound monitoring, `REINDEX CONCURRENTLY` and pg_repack for bloat.
- **Backup/DR**: pgBackRest incremental + PITR, with scheduled *restore* tests — not just backup tests.

3.3 MySQL / MariaDB at scale — the patterns

- **Topology & failover**: semi-sync or Group Replication / InnoDB Cluster; Orchestrator for topology management and automated failover; Vitess for horizontal sharding.
- **Proxy/pooling**: ProxySQL for read/write split, query routing rules and connection multiplexing.

- **Backup/DR**: Percona XtraBackup (hot physical backups) plus binlog for point-in-time recovery.
- **Online schema change**: gh-ost or pt-online-schema-change for lock-free ALTERs on large tables.
- **Tuning**: InnoDB buffer pool sizing, redo log, flush behaviour, and parallel replication workers.

Read/write splitting as configuration — version-controlled and deployed through the pipeline.

```
# ProxySQL: route reads to replicas, writes to the primary (hostgroups)
# Rule applied in order; first match wins.
mysql_query_rules:
  - rule_id: 10
    match_pattern: "^SELECT .* FOR UPDATE" # must hit primary
    destination_hostgroup: 10 # writer group
  - rule_id: 20
    match_pattern: "^SELECT" # plain reads
    destination_hostgroup: 20 # reader group (replicas)
```

3.4 Where AI plugs into the as-code loop

Stage	AI assist
PR / review	Summarise the migration; flag risky patterns (non-concurrent index, blocking lock, missing rollback); score change risk
CI	Detect query-plan regressions against a baseline before merge
Canary	Anomaly-detect latency/error/lag on the canary slice; recommend promote or roll back
Production	Triage alerts, assemble context, surface the matching runbook and similar past incidents
Capacity	Forecast growth and feed the recommended sizing into the next Terraform PR
Postmortem	Draft the timeline and contributing factors from telemetry and chat for human sign-off

3.5 Monitoring at fleet scale

One cluster is a dashboard; a fleet needs **aggregation**. Federate Prometheus (or use Thanos/Mimir/Cortex) for long-term, fleet-wide metrics; standardise one dashboard set rendered per cluster; roll SLOs up to a single reliability view; and put fleet-wide panels on the things that cause outages — replication health, backup/restore status, connection saturation and disk runway. The AI anomaly and forecast layer reads from this same store so humans and models share one source of truth.

- **Fleet overview**: every cluster's health, role, lag and budget on one screen.
- **SLO rollup**: error-budget burn per service, sorted by risk.
- **Backup & DR board**: last successful backup and last *restore* test per cluster.
- **Capacity & cost**: growth trends, runway-to-full, spend per cluster/team.

3.6 A day in the life — everything together

Morning. The AI digest summarises the night: backups green, one replica lag spike auto-recovered, the `billing` cluster forecast to hit 90% disk in ~19 days. You skim, agree, and let the disk forecast open a capacity ticket.

Midday. An app team's PR adds an index. CI runs the schema diff and the AI review flags it as a *non-concurrent* build on a 200M-row table. You require `CONCURRENTLY`, the author updates it, the pipeline canaries it on one replica, anomaly detection sees no regression, and GitOps promotes it to prod.

Afternoon. You turn the disk forecast into a Terraform PR resizing the `billing` volume; a teammate approves; Argo CD applies it — no console, no snowflake.

Evening on-call. A latency alert fires. The triage agent has already assembled the context — a long-running transaction blocking autovacuum, plus two similar past incidents and the matching runbook. You approve the proposed mitigation (terminate the offending session), latency recovers, and the postmortem draft is waiting for your review in the morning.

The throughline: reliability is engineered, not babysat. You define targets (SLOs), express the whole fleet as code (Terraform + operators + migrations + GitOps), monitor symptoms that map to user pain, and let AI carry the search-and-draft load so your judgment is spent where it counts — on the consequential decisions a machine shouldn't make alone.